

PRÁCTICA DIAGNÓSTICA · EVALUACIÓN INICIAL

Diagnóstico de Competencias en Java y Persistencia Básica

Prueba práctica individual en entorno de desarrollo para evaluar la base técnica previa a Acceso a Datos

MÓDULO PROFESIONAL Acceso a Datos (0486)	CICLO Y CURSO DAM — 2º Curso (2026-2027)	MODALIDAD Y ESPACIO Presencial / Distancia (Aula C210)
DOCENTE RESPONSABLE Sergio Capdevila Díez	TIEMPO ESTIMADO 120 - 150 minutos (2 sesiones)	CARÁCTER DE LA ACTIVIDAD Diagnóstica (Formativa, no computable)

 COMPETENCIAS Y CRITERIOS CURRICULARES DE REFERENCIA

- RA1: Manejo de Ficheros
- CE1.a (Clases de E/S)
- CE1.b (Flujos de Texto)
- CE1.c (Ficheros Binarios)
- RA2: Acceso a BBDD
- CE2.a (Sentencias SQL)

Esta práctica diagnóstica verifica el grado de soltura en la programación orientada a objetos en Java, el uso de colecciones y Streams, la gestión de excepciones y flujos de ficheros, y el dominio de sentencias SQL relacionales fundamentales para abordar con éxito las unidades de trabajo del módulo.

1. Contexto Técnico: Prototipo "LogiTech Express"

La empresa de logística urgente **LogiTech Express** requiere un prototipo de consola en **Java 21** para auditar sus operaciones diarias antes de conectar su sistema a una base de datos centralizada. Como desarrollador/a, debes crear una utilidad autónoma que gestione clientes y paquetes en memoria, guarde trazas en ficheros de texto (CSV), realice copias de seguridad de objetos en ficheros binarios y diseñe el esquema relacional de almacenamiento.

2. Enunciado y Tareas Paso a Paso

Tarea 1: Modelado POO, Colecciones y Java Streams (2.5 puntos)

- Clase `Cliente`** : Atributos privados `id` (String), `nombre` (String), `telefono` (String) y `activo` (boolean). Constructor completo, getters/setters, `toString()` y métodos `equals()` / `hashCode()` basados en `id` .
- Clase `Envio`** : Atributos privados `codigo` (String), `clienteId` (String), `pesoKg` (double), `coste` (double), `fecha` (LocalDate) y `entregado` (boolean). Encapsulación completa.
- Servicio `GestorLogistica`** :
 - Mantener una lista de envíos `List<Envio>` y un mapa de clientes `Map<String, Cliente>` (clave: `id`).
 - Implementar `List<Envio> obtenerEnviosPendientes()` utilizando **Java Streams** (`.stream().filter(e -> !e.isEntregado()).toList()`).
 - Implementar `double calcularFacturacionCliente(String clienteId)` mediante operaciones declarativas (`.filter(...).mapToDouble(Envio::getCoste).sum()`).

Tarea 2: Entrada/Salida en Ficheros de Texto (CSV) y Excepciones (2.5 puntos)

- Exportación a CSV**: Implementar `exportarEnviosCSV(String ruta)` para volcar la lista a `envios.csv` con encabezado:

```
codigo,clienteId,peso,coste,fecha,entregado
ENV-101,CLI-01,4.5,14.80,2026-09-15,false
ENV-102,CLI-02,1.2,6.50,2026-09-15,true
```

2. **Importación con try-with-resources:** Implementar `importarEnviosCSV(String ruta)` empleando `BufferedReader`. Validar cada línea y capturar `NumberFormatException` ante datos numéricos corruptos sin interrumpir la lectura del resto de registros.

Tarea 3: Serialización Binaria de Objetos (2.0 puntos)

1. Marcar la clase `Envio` con `Serializable` y declarar un `serialVersionUID = 1L` explícito.
2. Implementar `guardarBackupBinario(String ruta)` usando `ObjectOutputStream` para serializar la lista `List<Envio>` en `backup_envios.dat`.
3. Implementar `List<Envio> restaurarBackupBinario(String ruta)` usando `ObjectInputStream`, capturando de forma diferenciada `ClassNotFoundException` e `IOException`.

Tarea 4: Fundamentos de Bases de Datos SQL (2.0 puntos)

Redactar un archivo de script `logitech_schema.sql` con:

1. **DDL:** Sentencias `CREATE TABLE` para `CLIENTES` (PK: `id_cliente`) y `ENVIOS` (PK: `codigo_envio`, FK hacia clientes con integridad referencial, y restricción `CHECK (peso_kg > 0)`).
2. **DML:** Consulta `SELECT` con `INNER JOIN` para mostrar clientes con envíos superiores a 10.00 €, y una consulta con `GROUP BY` para calcular el importe total acumulado por cliente.

3. Entorno de Desarrollo y Política de Inteligencia Artificial

Entorno de Desarrollo Oficial

El entorno de trabajo principal es **Eclipse IDE** (Java SE 21 LTS). Se permite el uso de IntelliJ IDEA o VS Code siempre que el proyecto utilice estructura estándar de carpetas o Maven.

Política Estricta sobre Inteligencia Artificial (IA)

Al ser una **evaluación inicial diagnóstica**, está **terminantemente prohibido el uso de herramientas de IA generativa** (ChatGPT, GitHub Copilot, Claude, etc.). El objetivo exclusivo de esta prueba es identificar tu nivel real de autonomía técnica para planificar las adaptaciones docentes necesarias.

4. Rúbrica Analítica de Evaluación Diagnóstica (10.0 Puntos)

CRITERIO / INDICADOR	EXCELENTE (100%)	NOTABLE (75%)	APROBADO (50%)	INSUFICIENTE (<50%)
1. Modelado POO y Streams [2.5 Puntos]	Encapsulamiento perfecto, `equals/hashCode` correctos, uso elegante de Streams (`filter`, `sum`).	Clases y colecciones correctas. Streams operativos con pequeñas redundancias sintácticas.	Modelado básico funcional; utiliza bucles tradicionales `for-each` en lugar de Streams.	Falta de encapsulamiento, errores en POO o incapacidad de operar con listas/mapas.
2. Ficheros CSV y Excepciones [2.5 Puntos]	Uso riguroso de `try-with-resources`, parsing robusto de texto y captura selectiva de excepciones.	Lectura y escritura CSV funcionales con cierre seguro; aviso de error básico ante líneas corruptas.	Lee y escribe el fichero pero no gestiona el cierre seguro de recursos o falla ante errores leves.	Fuga de descriptores de fichero, excepciones no controladas o código que bloquea la ejecución.
3. Serialización Binaria [2.0 Puntos]	Implementa `Serializable` con `serialVersionUID`, serializa y deserializa la lista limpiamente.	Serialización y deserialización operativas; omite el `serialVersionUID` o detalles menores.	Guarda y recupera objetos pero con avisos de casting o sin captura de `ClassNotFoundException`.	No implementa `Serializable` o errores graves de E/S que impiden recuperar los datos.
4. Fundamentos SQL Relacionales [2.0 Puntos]	Script DDL con PK, FK y restricciones CHECK. Consultas SELECT con INNER JOIN y GROUP BY precisas.	DDL relacional correcto y funcional; consultas operativas con pequeños desajustes sintácticos.	DDL básico sin restricciones avanzadas; JOIN correcto pero dificultades en la agregación.	Errores en definición de claves foráneas o incapacidad para relacionar tablas en SQL.
5. Buenas Prácticas y Entorno [1.0 Punto]	Estructura modular en paquetes (`model`, `service`, `app`), nomenclatura estándar Java y Clean Code.	Proyecto ordenado y código limpio y legible.	Código en un único paquete sin modularizar, aunque compilable.	Código desordenado, sin indentación o con advertencias graves de compilación.
PUNTUACIÓN MÁXIMA TOTAL:				10.0 Puntos

5. Instrucciones de Entrega

- **Nombre del Proyecto:** Crear en Eclipse un proyecto llamado `EvaluacionInicial-NombreApellido`.
- **Estructura de Paquetes:** `es.cifpaviles.logitech.model`, `es.cifpaviles.logitech.service` y `es.cifpaviles.logitech.app`.
- **Archivo SQL:** Incluir el script `logitech_schema.sql` en la raíz del proyecto.
- **Vía de Entrega:** Subir el archivo comprimido `.zip` en la tarea habilitada de Moodle Educastur o en el repositorio Git indicado en clase.